

Proxy OPD: On-Policy Distillation with Transferable Relative Proxy Update

Daocheng Fu^{*,1,2}, Rong Wu^{*,1,3}, Yu Yang^{*,1,3}, Jianbiao Mei¹,
Licheng Wen^{1,4,5}, Pinlong Cai¹, Xuemeng Yang^{†,1}, Yong Liu³, Botian Shi^{†,1}, Yu Qiao¹

¹ Shanghai AI Laboratory, ² Fudan University, ³ Zhejiang University,

⁴ Shanghai Innovation Institute, ⁵ Shanghai Jiao Tong University

* Equal Contribution, † Corresponding Author

Abstract

Post-training for large language models typically couples policy exploration with model optimization, hindering the reuse of high-reward behaviors from policy exploration. While on-policy distillation alleviates this by consolidating independently optimized experts, its reliance on matching absolute expert distributions can yield suboptimal supervision, especially when the target model possesses a different prior or already surpasses the expert’s capabilities. To alleviate this, we introduce **Proxy OPD** (P-OPD), an asynchronous post-training framework that transfers reward-induced policy improvements rather than absolute policy distributions. P-OPD first optimizes a proxy policy via reward feedback. It then extracts the relative distributional changes between the proxy’s initial and optimized states, transferring these directional updates through the target model’s own on-policy trajectories while retaining the target policy as the reference. This decoupled formulation requires the proxy to provide merely a useful direction of improvement rather than superior absolute capability, enabling update signals from older or weaker proxies to remain highly effective. Systematic experiments on Qwen3-family models across mathematical reasoning and code generation demonstrate that P-OPD consistently enhances already strong target models. Furthermore, transfer intensity can be dynamically modulated through signal scaling, making the extracted update signals seamlessly reusable across diverse model variants and training configurations. These results establish relative policy updates as highly reusable, adjustable assets for scalable, reward-based post-training.



Date: August 10, 2026



GitHub Repo: <https://github.com/KnowledgeXLab/P-OPD>



Huggingface Page: https://huggingface.co/KnowledgeXLab/P-OPD_Experiments

1 Introduction

Post-training is essential for improving the usability and task-specific capabilities of large language models (LLMs) [1]. Policy-based reinforcement learning (RL) approaches, such as PPO [2] and GRPO [3], achieve this goal by optimizing models with on-policy rollouts guided by reward feedback. However, this direct optimization approach inherently couples policy exploration with individual model training: adapting a new model requires collecting new trajectories and rerunning reward optimization from scratch. Consequently, high-reward behaviors discovered in previous runs are difficult to transfer or reuse, while jointly optimizing heterogeneous domains may introduce cross-domain interference (Fig. 1(a)).

Recently, many research efforts have adopted on-policy distillation (OPD) [4, 5] as an alternative post-training strategy. Compared with direct RL optimization, OPD provides a more flexible paradigm for developing and reusing specialized capabilities. OPD-based pipelines typically consist of two stages: *policy exploration* and *policy alignment*. During exploration, policies are independently optimized with task-specific data and reward signals to obtain specialized experts. During alignment, their capabilities are consolidated into a target model through

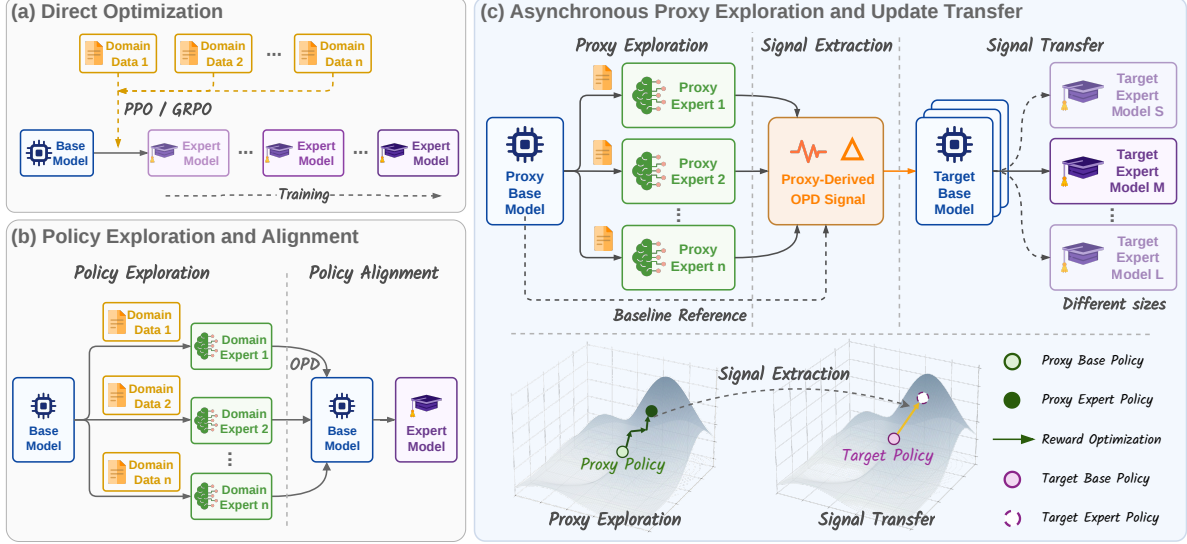


Figure 1: **Comparison of post-training pipelines.** (a) **Direct Optimization:** The target model is optimized directly using reward feedback. (b) **Policy Exploration and Alignment:** Policies are independently optimized into experts and then used to align the target model through distribution matching. (c) **Asynchronous Proxy Exploration and Update Transfer (Ours):** Proxy policies are explored independently, with their relative on-policy updates extracted and transferred to the target model.

distribution matching (Fig. 1(b)). This design enables parallel expert construction, reduces optimization interference, and allows experts to be cached and reused [6, 7, 8].

However, existing OPD pipelines typically perform policy alignment by matching the target model to the *absolute output distribution* of each optimized expert. The effectiveness of this process depends on whether the expert distribution remains a suitable supervision target for the target model. When the target model approaches or surpasses the expert, or develops a substantially different prior distribution, directly matching the expert may provide diminishing learning signals and even constrain behaviors that are already superior in the target model. Consequently, expert selection and reuse, especially across model generations, are constrained by the expert’s absolute capability and compatibility with the target model. This observation suggests a new perspective: *instead of distilling the optimized policy itself, can we transfer the relative policy-improvement signal produced during expert optimization?*

To this end, we propose **Proxy OPD (P-OPD)**, a new OPD-style post-training strategy based on *relative update transfer*. Instead of matching the target model to the absolute distribution of a specific expert, our strategy selects a proxy model from which to extract and transfer the policy-improvement signal induced by reward optimization. Our strategy consists of three stages: **proxy exploration**, **update-signal extraction**, and **signal transfer**. First, the proxy model is optimized with reward feedback to discover high-reward behaviors. We then compare the initial and optimized proxy policies to extract their relative distributional changes, capturing how reward optimization transforms the policy rather than the final policy distribution itself. Finally, these relative changes are transferred to the target model through its own on-policy trajectories, guiding alignment while preserving the target model’s original distribution as the reference. Unlike absolute expert distillation, our approach does not require the proxy model to surpass the target model in absolute capability; it only needs to provide a useful direction of policy improvement. This enables update signals extracted from older or weaker proxies to remain effective, while allowing proxy exploration to proceed independently of target-model training. The resulting proxy pairs can be cached as reusable exploration assets and transferred across model generations, yielding an asynchronous post-training pipeline (Fig. 1(c)).

We systematically evaluate P-OPD on Qwen3-family models [9] across mathematical reasoning and code generation tasks. Empirical results demonstrate that relative update transfer consistently enhances even highly capable target models, confirming that valuable policy-improvement signals remain robustly transferable beyond their source policies. Furthermore, signal scaling offers precise control over transfer intensity, enabling the seamless reuse of proxy exploration assets across diverse model variants and training configurations. Collectively, these findings establish relative policy updates as reusable, adjustable, and highly scalable assets for reward-based post-training.

Our main contributions are as follows:

- We introduce *relative update transfer* for LLM post-training, shifting from matching absolute expert distributions to transferring reward-induced policy improvements.
- We propose **Proxy OPD**, which extracts relative distributional changes from a proxy policy and transfers them through the target model’s own on-policy trajectories.
- We demonstrate that P-OPD consistently improves well-performing target models, with controllable transfer strength and reusable update signals across model variants and training configurations.

2 Related Works

Post-training plays a crucial role in improving large language models, and recent studies have proposed various algorithms to enhance training efficiency and model performance [1]. We summarize representative methods into two categories: reward optimization and distribution matching.

2.1 Reward Optimization

Conventional reward-optimization methods improve policies using feedback from reward models [10, 11, 12]. Recent work has focused on making reward signals more fine-grained, moving from sequence-level rewards to step-wise [13, 14, 15] and token-level rewards [16, 17], which can provide more precise supervision. However, training reliable reward models requires large-scale high-quality data. AI-generated feedback [18, 19] reduces this cost, but most methods remain PPO-like [2] and require multiple models during training. GRPO [3] lowers the overhead by estimating advantages through group-wise relative comparisons. Nevertheless, reward-optimization methods still depend on on-policy autoregressive sampling and reward evaluation for each trajectory.

2.2 Distribution Matching

Distribution-matching methods avoid explicit reward optimization by directly aligning the model with a target distribution. SFT [20] and KD [21, 22, 23, 24, 25] align student models with hard or soft targets, while DPO [26] constructs an implicit target distribution from preference pairs. These offline methods are efficient but may suffer from distribution shift. OPD [4, 27, 28, 5, 29, 30] mitigates this issue by matching the teacher distribution on trajectories sampled from the student. Although it still requires autoregressive sampling, distribution matching often provides more direct credit assignment and faster convergence. However, these methods rely on access to a stronger target distribution, which can introduce substantial hidden costs.

Both reward optimization and distribution matching depend on high-quality target distributions. As base models become stronger, obtaining such distributions becomes increasingly costly. To improve post-training scalability, we propose to decouple model optimization from reward exploration by extracting, storing, and reusing training signals asynchronously, thereby improving training efficiency.

3 Preliminary Analysis: Reward Optimization vs. Distribution Matching

As discussed in Section 1, parallel post-training can be viewed as comprising two complementary stages: policy exploration and policy alignment. Reward optimization primarily facilitates exploration by discovering high-reward behaviors, whereas distribution matching aligns a model with a given target distribution. To better understand their respective roles and training efficiencies, we compare two representative methods: GRPO and OPD.

Using Qwen3-1.7B as the base model, we evaluate four training configurations. First, we train a standard GRPO model for 500 steps, using it both as a reward-optimization baseline and as the expert teacher for OPD. Standard OPD then aligns a student model with this teacher. To examine whether incorporating reward information changes the behavior of OPD, we further consider two reward-informed variants: **OPD FT (Filtered Trajectories)**, which excludes training groups in which all student trajectories are correct, and **OPD URM**

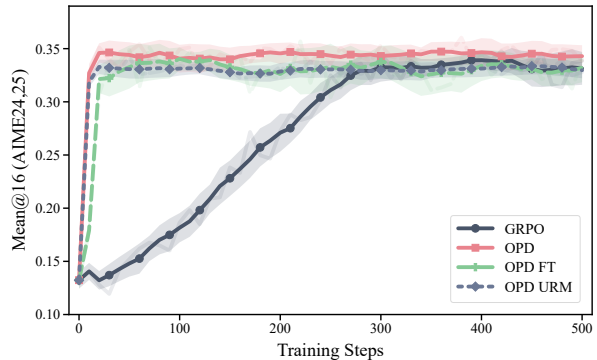


Figure 2: Mean@16 trajectories on AIME24 and AIME25. OPD variants converge within 70 steps and are extended to 500 steps based on converged statistics (mean and std)

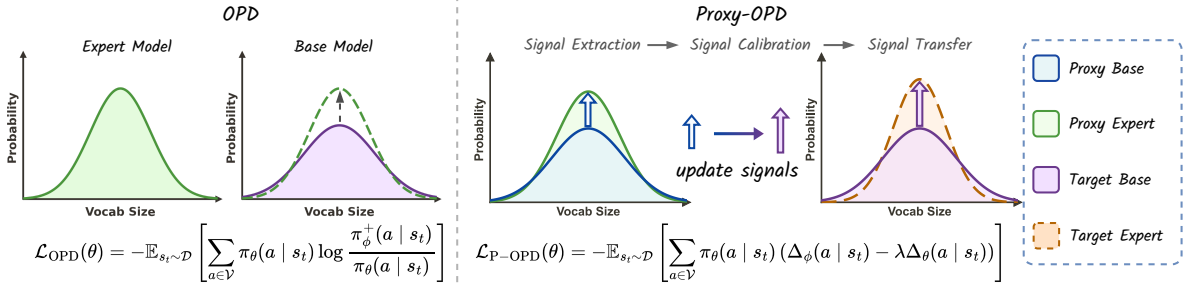


Figure 3: **Illustration of the proposed method.** **Left:** OPD directly aligns the token distributions for samples generated by the target policy. **Right:** Proxy-OPD first extracts relative update signals at the sampled tokens, then calibrates these signals, and finally applies the calibrated signals to the current token distribution.

(**Update Reward Mask**), which masks token updates whose directions conflict with the trajectory-level reward signal.

As shown in Fig. 2, OPD and its variants converge substantially faster than GRPO, while all methods attain statistically comparable final performance. Neither FT nor URM provides a significant improvement over standard OPD. These results indicate that the reward-informed modifications do not introduce an effective learning signal beyond that already encoded in the teacher distribution. OPD therefore remains primarily a distribution-matching procedure that efficiently transfers the teacher’s behavior rather than independently discovering higher-reward policies.

Taken together, these observations highlight the complementary roles of reward optimization and distribution matching. Distribution matching enables efficient policy alignment once a high-quality target distribution is available, whereas obtaining such a target still requires reward-driven policy exploration. In this sense, reward optimization discovers useful directions for policy improvement, while distribution matching transfers the resulting behaviors to the student. This distinction motivates us to investigate how such update signals can be decoupled from exploration, reused, and efficiently transferred across models.

4 Methodology

Distribution matching provides an efficient way to convert reward-driven policy improvement into direct policy alignment. In our setting, a natural instantiation is to first optimize a proxy model with rewards and then distill the target model toward the optimized proxy. However, this direct formulation transfers the proxy’s absolute output distribution, making the training signal strongly tied to the proxy policy itself. When the target model becomes close to, or even surpasses, the optimized proxy in some states, direct imitation provides limited benefit and may even suppress better token preferences already possessed by the target model.

To address this limitation, we reformulate distillation as *relative update-signal transfer*. Instead of aligning the target model to the optimized proxy distribution, we extract how reward optimization changes the proxy and transplant this change onto the target model’s own base distribution. Our method consists of three stages: *proxy exploration*, *update-signal extraction*, and *signal transfer*.

4.1 Setup

Let x denote an input prompt and $y = (y_1, \dots, y_T)$ denote a generated response. At decoding step t , we define the token-level state as

$$s_t = (x, y_{<t}). \quad (1)$$

Our framework involves two models sharing a common vocabulary $\mathcal{V}$: a proxy model used for reward exploration, and a target model that we ultimately aim to improve. We use the following four policies throughout this section: (1) **Proxy base policy** π_ϕ : the proxy policy before reward optimization; (2) **Proxy expert policy** π_ϕ^+ : the optimized proxy policy obtained after reward optimization; (3) **Target base policy** π_{ref} : the frozen initial policy of the target model; (4) **Target expert policy** π_θ : the trainable target policy to be optimized.

Here, π_ϕ^+ is not assumed to be given in advance. It is produced from π_ϕ during the proxy exploration stage, while π_{ref} remains fixed as the anchor for training π_θ .

4.2 OPD as a Direct Baseline

A straightforward way to use the proxy expert policy π_ϕ^+ is on-policy distillation (OPD), which directly aligns the target expert policy π_θ to the proxy expert distribution:

$$\mathcal{L}_{\text{OPD}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \left[D_{\text{KL}} \left(\pi_\theta(\cdot | x) \parallel \pi_\phi^+(\cdot | x) \right) \right]. \quad (2)$$

At the token level, this objective can be written as

$$\mathcal{L}_{\text{OPD}}(\theta) = -\mathbb{E}_{s_t \sim \mathcal{D}} \left[\sum_{a \in \mathcal{V}} \pi_\theta(a | s_t) \log \frac{\pi_\phi^+(a | s_t)}{\pi_\theta(a | s_t)} \right]. \quad (3)$$

Although simple, OPD directly uses the distribution of π_ϕ^+ as the alignment target. For an already capable target model, further improvement requires this target to encode consistently better token preferences, placing a stringent requirement on the proxy expert. This motivates us to generalize OPD from matching an absolute proxy expert distribution to transferring a relative reward-induced update. The following sections present this process in three stages.

4.3 Stage 1: Proxy Exploration

The goal of proxy exploration is to obtain the proxy expert policy π_ϕ^+ from the proxy base policy π_ϕ . Direct reward optimization on the target model can be expensive and inflexible, especially when adapting to different reward functions. We therefore decouple reward exploration from target-model training by using the proxy model as an efficient exploration testbed.

Given a reward function $r(x, y)$, we optimize the proxy base policy π_ϕ using standard reward optimization algorithms, such as PPO or GRPO:

$$\pi_\phi^+ \leftarrow \text{RewardOpt}(\pi_\phi, r). \quad (4)$$

The resulting proxy expert policy π_ϕ^+ captures reward-preferred behavioral changes discovered through trial-and-error exploration.

This stage avoids repeatedly running costly reward optimization on the target model. It also allows multiple proxy base policies to be optimized in parallel under different rewards or data distributions, producing diverse proxy expert policies whose update signals can be extracted and reused for subsequent transfer.

4.4 Stage 2: Update-Signal Extraction

After proxy exploration, standard distillation would directly use the proxy expert policy π_ϕ^+ as the target. In contrast, we extract the *relative* change from the proxy base policy π_ϕ to the proxy expert policy π_ϕ^+ .

For each state s_t and token $a \in \mathcal{V}$, we define the proxy-induced update signal as

$$\Delta_\phi(a | s_t) = \log \pi_\phi^+(a | s_t) - \log \pi_\phi(a | s_t) = \log \frac{\pi_\phi^+(a | s_t)}{\pi_\phi(a | s_t)}. \quad (5)$$

Crucially, Δ_ϕ is a log-ratio rather than an absolute probability distribution. It therefore represents a transferable update direction instead of a proxy-specific target distribution. We further introduce a scaling coefficient $\alpha > 0$ and use $\alpha \Delta_\phi$ as the transferred signal, where larger α amplifies the proxy-discovered update and smaller α yields more conservative transfer.

4.5 Stage 3: Signal Transfer

We now transfer the extracted proxy update signal to the target model. The proxy signal Δ_ϕ indicates how reward optimization changes the proxy policy at each state-token pair. However, directly applying this signal to the target model may be overly aggressive, since the proxy and target models can have different base distributions and different capabilities. Therefore, signal transfer should both encourage the target policy to follow the proxy-induced update direction and regularize it against excessive deviation from its own base policy.

To this end, we optimize the target expert policy π_θ with two terms. The first term rewards tokens that are unweighted by proxy reward optimization, measured by $\Delta_\phi(a | s_t)$. The second term penalizes the target policy change relative

to its frozen base policy π_{ref} . Specifically, we define the target-policy change as

$$\Delta_{\theta}(a \mid s_t) = \log \frac{\pi_{\theta}(a \mid s_t)}{\pi_{\text{ref}}(a \mid s_t)}. \quad (6)$$

We then formulate the signal-transfer objective as

$$\max_{\theta} \mathbb{E}_{s_t \sim \mathcal{D}} \left[\sum_{a \in \mathcal{V}} \pi_{\theta}(a \mid s_t) (\alpha \Delta_{\phi}(a \mid s_t) - \beta \Delta_{\theta}(a \mid s_t)) \right], \quad (7)$$

where $\alpha > 0$ controls the strength of the transferred proxy update signal, and $\beta > 0$ controls the conservatism of the target policy with respect to its own base policy. A larger α makes the target model more strongly absorb the proxy-discovered reward direction, while a larger β discourages large deviations from π_{ref} .

Since multiplying the objective by a positive constant does not change its optimizer, we divide Eq. 7 by α and define

$$\lambda = \frac{\beta}{\alpha}. \quad (8)$$

This yields the calibrated token-level utility

$$r_{\lambda}(a \mid s_t) = \Delta_{\phi}(a \mid s_t) - \lambda \Delta_{\theta}(a \mid s_t). \quad (9)$$

Accordingly, the final signal-transfer loss is

$$\mathcal{L}_{\text{proxy}}(\theta) = -\mathbb{E}_{s_t \sim \mathcal{D}} \left[\sum_{a \in \mathcal{V}} \pi_{\theta}(a \mid s_t) (\Delta_{\phi}(a \mid s_t) - \lambda \Delta_{\theta}(a \mid s_t)) \right]. \quad (10)$$

Under this parameterization, λ is the only effective calibration coefficient. It determines how conservatively the target model absorbs the proxy-induced update signal. A larger λ imposes a stronger penalty on deviations from the base policy and therefore leads to more conservative updates, whereas a smaller λ allows the target policy to more aggressively follow the reward-induced direction discovered by the proxy.

During signal transfer, the proxy base policy π_{ϕ} , the proxy expert policy π_{ϕ}^+ , and the target base policy π_{ref} remain frozen. Gradients are applied only to the target expert policy π_{θ} .

Overall, our method preserves the online distribution-matching nature of OPD, but replaces direct imitation of the proxy expert distribution with relative update-signal transfer. The proxy model is used to discover how reward optimization changes a policy, while the target model absorbs this change in its own base-policy-centered space.

5 Experiments

In this section, we design experiments to answer the following research questions:

- Can proxy-explored update signals improve the target model? (Section 5.1)
- Can these signals transfer across scales, stages, and model generations? (Section 5.2)
- How do proxy-explored signals compare with target-explored signals? (Section 5.3)

5.1 Performance Improvement

To evaluate the effectiveness of P-OPD, we conduct update signal transfer experiments in the Math and Code domains. For Math, we train the proxy base model Qwen3-4B with GRPO for 500 steps on DeepMath-103K [31]¹, producing the proxy expert Qwen3-4B Math (RL). Similarly, for Code, we train Qwen3-4B with GRPO for 300 steps on Eurys-RL-Code [32], producing Qwen3-4B Code (RL). We then transfer the learned update signals to the target base model Qwen3-8B, using a calibration coefficient of $\lambda = 1.0$ for P-OPD (4B→8B).

As shown in Table 1, the transferred update signals consistently improve Qwen3-8B across all evaluated benchmarks in both domains. Notably, on AIME24, HMMT25 (Nov), and LCB [33], the gains achieved by the target model even exceed those obtained during proxy exploration. Although OPD also improves the target base model by directly aligning it with the absolute output distribution of the proxy expert, its overall performance remains less competitive. In contrast, P-OPD transfers relative update signals and achieves stronger results on most benchmarks, as well as better average performance in both domains.

¹We filter the dataset using the selection strategy proposed in prior work [28].

Table 1: **Mean results of math and code benchmarks.** Math benchmarks report Mean@16 results, while code benchmarks report Mean@8 results. Absolute scores and gains (Δ) over respective base models are reported in parentheses. The best absolute scores are highlighted in **bold**. **Bold+Teal** indicates the best gains. In OPD and P-OPD, “ $\rightarrow$ ” indicates that update signals are extracted from the proxy expert model ($\hat{\mu}$) on the left and transferred to the target model (μ) on the right.

Model	Math Benchmarks					Code Benchmarks			
	AIME24	AIME25	HMMT25 (Feb)	HMMT25 (Nov)	AVG.	HumanEval+	MBPP+	LCB	AVG.
<i>Proxy Exploration</i>									
Qwen3-4B (Base)	23.2	22.1	10.6	7.9	16.0	79.4	64.1	18.0	53.8
Qwen3-4B (RL)	58.8 (+35.6)	55.8 (+33.7)	33.1 (+22.5)	38.3 (+30.4)	46.5 (+30.5)	82.5 (+3.1)	68.7 (+4.6)	19.0 (+1.0)	56.7 (+2.9)
<i>Update Signal Transfer</i>									
Qwen3-8B (Base)	24.4	22.5	12.3	10.0	17.3	80.5	70.6	16.7	55.9
OPD (4B $\hat{\mu} \rightarrow \mu$ 8B)	59.6 (+35.2)	50.6 (+28.1)	27.5 (+15.2)	36.7 (+26.7)	43.6 (+26.3)	84.2 (+3.7)	71.8 (+1.2)	21.5 (+4.8)	59.2 (+3.3)
P-OPD (4B $\hat{\mu} \rightarrow \mu$ 8B)	62.5 (+38.1)	52.7 (+30.2)	34.2 (+21.9)	40.4 (+30.4)	47.5 (+30.2)	83.1 (+2.6)	73.5 (+2.9)	25.0 (+8.3)	60.5 (+4.6)

Table 2: **Transferability of proxy update signals.** Mean@16 results of math benchmarks. P-OPD (4B $\hat{\mu} \rightarrow \mu$ 8B) uses proxy base model Qwen3-4B to explore update signals, which are then transferred sequentially to Qwen3-1.7B and Qwen3-8B.

Model	AIME24	AIME25	HMMT25 (Feb)	HMMT25 (Nov)	Avg.
<i>1.7B Parameter Scale Exploration</i>					
Qwen3-1.7B (Base)	11.9	11.0	5.8	5.2	8.5
Qwen3-1.7B (RL)	35.6 (+23.7)	31.7 (+20.7)	19.6 (+13.8)	15.2 (+10.0)	25.5 (+17.0)
<i>4B Parameter Scale Exploration</i>					
Qwen3-4B (Base)	23.2	22.1	10.6	7.9	16.0
Qwen3-4B (RL)	58.8 (+35.6)	55.8 (+33.7)	33.1 (+22.5)	38.3 (+30.4)	46.5 (+30.5)
<i>8B Parameter Scale Transfer</i>					
Qwen3-8B (Base)	24.4	22.5	12.3	10.0	17.3
OPD (1.7B $\hat{\mu} \rightarrow \mu$ 8B)	41.0 (+16.7)	34.4 (+11.9)	19.2 (+6.9)	18.5 (+8.5)	28.3 (+11.0)
OPD (4B $\hat{\mu} \rightarrow \mu$ 8B)	59.6 (+35.2)	50.6 (+28.1)	27.5 (+15.2)	36.7 (+26.7)	43.6 (+26.3)
P-OPD (1.7B $\hat{\mu} \rightarrow \mu$ 8B)	50.8 (+26.4)	42.1 (+19.6)	22.9 (+10.6)	30.6 (+20.6)	36.6 (+19.3)
P-OPD (4B $\hat{\mu} \rightarrow \mu$ 8B)	62.5 (+38.1)	52.7 (+30.2)	34.2 (+21.9)	40.4 (+30.4)	47.4 (+30.1)
P-OPD (4B $\hat{\mu} \rightarrow \mu$ 1.7B $\hat{\mu} \rightarrow \mu$ 8B)	60.6 (+36.2)	49.6 (+27.1)	28.5 (+16.2)	36.7 (+26.7)	43.9 (+26.6)

5.2 Transferability of Update Signals

We further evaluate proxy update signal transfer across different model scales to investigate its transferability and characterize its behavior.

5.2.1 Cross-Scale and Sequential Transfer

We conduct cross-scale transfer experiments within the Qwen3 model family. Specifically, we train Qwen3-1.7B and Qwen3-4B as proxy base models on DeepMath-103K using GRPO for 500 steps, obtaining the corresponding proxy experts Qwen3-1.7B (RL) and Qwen3-4B (RL). We then apply OPD and P-OPD to Qwen3-8B, where OPD directly aligns the target model with the absolute output distribution of the proxy expert, while P-OPD transfers the relative update signals learned during proxy exploration. To account for the scale differences between the proxy and target models, we set the calibration coefficient to $\lambda = 1.5$ for the 1.7B $\rightarrow$ 8B transfer and $\lambda = 1.0$ for the 4B $\rightarrow$ 8B transfer.

As shown in Table 2, update signals explored by both the 1.7B and 4B proxy models transfer effectively to the 8B target model. In both cases, P-OPD substantially improves the target model, with average gains comparable to or even greater than those achieved during proxy exploration. In contrast, OPD consistently yields smaller improvements across all benchmarks, demonstrating the advantage of transferring relative update signals over directly matching the proxy expert’s absolute output distribution.

We further investigate whether update signals remain effective after multiple rounds of transfer. Specifically, we first transfer the update signals explored by Qwen3-4B to Qwen3-1.7B and then transfer them from Qwen3-1.7B to Qwen3-8B, using $\lambda = 1.0$ at both stages. Despite this sequential transfer, Qwen3-8B still achieves substantial performance gains. Moreover, the resulting improvement approaches that of the direct 4B $\rightarrow$ 8B transfer, indicating that the explored update signals are largely preserved across multiple rounds of transfer.

5.2.2 Sensitivity to the Calibration Coefficient

Transferring update signals across model scales requires an appropriate calibration coefficient λ to control the signal strength. We conduct sensitivity analyses for transfers from Qwen3-4B and Qwen3-1.7B by varying the calibration coefficient². Due to the computational cost, we evaluate a selected set of coefficients for each proxy model and fit a cubic polynomial to the resulting discrete measurements to estimate the optimal coefficient.

As shown in Fig. 4, the fitted optimal coefficients are $\lambda^* = 1.08$ for Qwen3-4B and $\lambda^* = 1.57$ for Qwen3-1.7B. Moreover, Qwen3-4B achieves a higher peak score than Qwen3-1.7B, indicating that proxy models of different scales may explore update signals of different quality.

Although polynomial fitting provides an effective calibration coefficient for each proxy model, the relationship between the proxy and target models may vary across sampled states. Developing adaptive, potentially state-dependent calibration strategies therefore represents an important direction for future work.

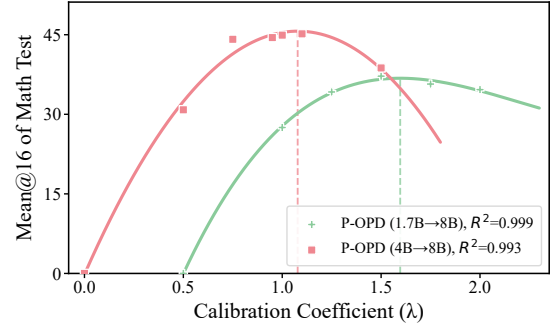


Figure 4: **Sensitivity to proxy model scale and calibration coefficient.** Each curve is obtained by fitting a cubic polynomial to the experimental results. An R^2 value closer to 1 indicates a better fit.

5.2.3 Cross-Generation Reusability

Transferring update signals provides a natural way to reuse signals explored by earlier model generations to improve newer models. To evaluate this cross-generation reusability, we conduct transfer experiments between Qwen2.5-7B [34] and Qwen3-8B. Skywork-OR1-Math-7B [35] is an expert model derived from Qwen2.5-7B through post-training on multiple datasets with multiple training methods. We therefore treat Qwen2.5-7B and Skywork-OR1-Math-7B as the proxy base and expert models, respectively. For comparison, we also obtain a current-generation expert, Qwen3-8B (RL), by training Qwen3-8B (Base) on DeepMath-103K with GRPO for 400 steps.

As shown in Table 3, update signals explored by the older-generation model effectively improve the newer-generation target models. Moreover, transferring these signals to Qwen3-8B yields a greater improvement than directly exploring update signals with Qwen3-8B itself. In contrast, OPD, which directly aligns the target model with the absolute output distribution of the older-generation expert, performs substantially worse than P-OPD. These results demonstrate that relative update signals remain reusable across model generations and can provide valuable guidance for improving newer models.

Table 3: **Cross-generation reusability of transferrable update signals.** Mean@16 results of math benchmarks. “SW7B” and “Q8B” denote Skywork-OR1-Math-7B (proxy expert 🎓) and Qwen3-8B (target student 🧑), respectively. Update signals explored by an older-generation proxy can effectively guide the training of a new-generation target model.

Model	AIME24	AIME25	HMMT25 (Feb)	HMMT25 (Nov)	AVG.
<i>Cross-generation Base Models</i>					
Qwen2.5-7B (Base)	4.4	0.8	0	1.5	1.7
Qwen3-8B (Base)	24.4	22.5	12.3	10.0	17.3
<i>Obtaining Update Signals</i>					
Skywork-OR1-Math-7B	63.5 (+59.1)	48.5 (+47.7)	30.8 (+30.8)	37.7 (+36.2)	45.2 (+43.5)
Qwen3-8B Math (RL)	65.2 (+40.8)	53.8 (+31.3)	29.8 (+17.5)	29.4 (+19.4)	44.5 (+27.2)
<i>Update Signal Transferred Across Generations</i>					
OPD (SW7B 🎓 → Q8B)	60.4 (+36.0)	48.8 (+26.3)	29.0 (+16.7)	38.1 (+28.1)	44.1 (+26.8)
P-OPD (SW7B 🎓 → Q8B)	69.2 (+44.8)	57.5 (+35.0)	35.0 (+22.7)	48.1 (+38.1)	52.5 (+35.2)

5.3 Comparison of Exploration Quality

Proxy-guided transfer offloads update-signal exploration to smaller models, but the explored signals may differ in quality from those of the target model. To quantify this difference, we use Qwen3-8B as the target model and Qwen3-1.7B and Qwen3-4B as proxies with varying exploration budgets. In Figure 5 (left), GRPO (8B-RL400) denotes direct exploration by Qwen3-8B for 400 steps, while P-OPD (1.7B-RL500→8B) denotes 500-step exploration by Qwen3-1.7B followed by signal transfer to Qwen3-8B. Other settings follow the same notation.

Signal transfer converges much faster than direct exploration. Under similar exploration budgets (400 steps for Qwen3-8B versus 500 steps for the proxy models), Qwen3-8B explores higher-quality signals than the smaller

²We report the average performance across all test sets here. Results on individual test sets are provided in “Technical Supplement: Experiment Details.”

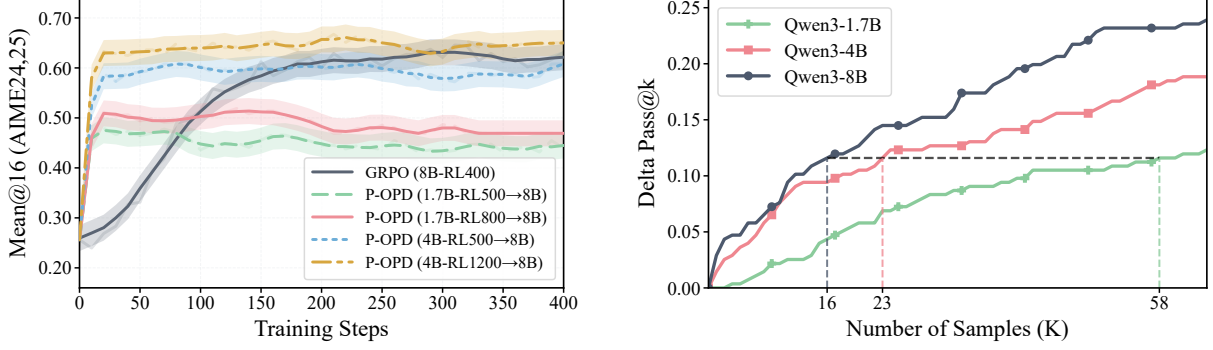


Figure 5: Left: **Mean@16 trajectories on AIME2024 and AIME2025**. All P-OPD runs converge within 70 steps and are extended using converged-region statistics. Right: **$\Delta\text{Pass}@K$ ($\text{Pass}@K - \text{Pass}@1$) versus the sampling budget K** . More samples consistently uncover additional correct solutions.



Figure 6: Revisiting the relationship between data/environment and models. By decoupling the exploration process from the model, update signals can be stored and reused independently. To maximally compress the useful information in the data, optimizing the exploration strategy of proxy models becomes a valuable research direction. Moreover, the adaptive correction of update signals during transfer is crucial, as it directly affects transfer efficiency.

proxies. However, increasing the proxy exploration budget narrows this gap. With 1,200 exploration steps, Qwen3-4B yields transferred performance comparable to or better than direct exploration by Qwen3-8B. The omitted P-OPD (4B-RL1200→8B) run converges slightly above GRPO (8B-RL400).

Figure 5 (right) further illustrates the gains obtained under varying exploration budgets. As the sampling budget increases, smaller models discover additional useful solutions and can eventually match the exploration quality of larger models. This observation broadens the potential applications of asynchronous update-signal exploration: appropriately scaled proxy models can be paired with more flexible exploration strategies to obtain high-quality update signals. Combined with the cross-generation reusability of these signals, this paradigm enables an “explore once, transfer multiple times” workflow, thereby improving the efficiency of post-training.

6 Conclusion

We introduced **Proxy OPD (P-OPD)**, an OPD-style post-training strategy that transfers reward-induced relative policy updates instead of matching an expert’s absolute output distribution. This design decouples proxy exploration from target-model training and allows the resulting update signals to be reused across models. Experiments on mathematical reasoning and code generation show that P-OPD consistently improves strong target models and generally outperforms conventional OPD. The update signals transfer effectively across model scales and generations, remain useful after sequential transfer, and can be adjusted through a calibration coefficient. We also find that increasing the exploration budget enables smaller proxies to discover signals comparable to those explored directly by larger models. These results support an asynchronous “explore once, transfer multiple times” post-training pipeline.

7 Discussion and Future Works

P-OPD provides a novel lens to revisit the relationship between training data and policy models. As illustrated in Fig. 6, it abstracts post-training into two sequential processes: extracting environmental data into a transferable signal, and subsequently applying this signal to a target primary model.

Update-Signal Extraction (Data / Environment $\rightarrow$ Transferable Update Signal):

- **Mechanism:** This stage utilizes a proxy model to explore the environment via reward optimization. It functions as task-specific information compression, distilling useful supervision from the data into reusable distributional improvement signals.
- **Bottleneck:** Signal quality is intrinsically bounded by the data distribution, the proxy exploration strategy, and the signal aggregation mechanism. Optimizing these factors, e.g., via multi-model parallel exploration or ensemble multi-source voting, can significantly raise this upper bound.
- **Advantage:** Ultimately, this phase unlocks the highly efficient paradigm of “compressing once and reusing many times.”

Adaptive Signal Transfer (Update Signal $\rightarrow$ Primary Model):

- **Mechanism:** This stage transfers the extracted signal to the primary model to realize policy optimization.
- **Bottleneck:** The primary challenge is the distributional gap between the proxy and primary models. Although a constant calibration coefficient provides a solid baseline, variations in the dynamic trajectory, such as logit overlap and local entropy, significantly affect transfer efficiency. Overcoming this requires trajectory-adaptive calibration methods that dynamically adjust transfer intensity based on local distributional relationships.
- **Advantage:** Despite these challenges, this mechanism successfully enables robust cross-model transfer and high signal reusability.

Overall, our framework demonstrates the theoretical and practical feasibility of asynchronous exploration and the reusable transfer of update signals. We leave the design of stronger *proxy exploration strategies* and more sophisticated *adaptive calibration mechanisms* as promising directions for future research.

References

- [1] Hanyu Lai, Xiao Liu, Junjie Gao, Jiale Cheng, Zehan Qi, Yifan Xu, Shuntian Yao, Dan Zhang, Jinhua Du, Zhenyu Hou, et al. A survey of post-training scaling in large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2771–2791, 2025.
- [2] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [3] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [4] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, volume 2024, pages 21246–21263, 2024.
- [5] Mingyang Song and Mao Zheng. A survey of on-policy distillation for large language models. *arXiv preprint arXiv:2604.00626*, 2026.
- [6] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026.
- [7] Bangjun Xiao, Bingquan Xia, Bo Yang, Bofei Gao, Bowen Shen, Chen Zhang, Chenhong He, Chiheng Lou, Fuli Luo, Gang Wang, et al. Mimo-v2-flash technical report. *arXiv preprint arXiv:2601.02780*, 2026.
- [8] Lei Bai, Zongsheng Cao, Yang Chen, Zhiyao Cui, Shangheng Du, Yue Fan, Shiyang Feng, Zijie Guo, Haonan He, Liang He, Xiaohan He, Shuyue Hu, Yusong Hu, Songtao Huang, Yichen Jiang, Hao Li, Xin Li, Dahua Lin, Weihao Lin, Fenghua Ling, Dongrui Liu, Zhuo Liu, Runmin Ma, Chunjiang Mu, Haoyang Peng, Tianshuo Peng, Jinxin Shi, Luohe Shi, Boyuan Sun, Zelin Tan, Shengji Tang, Qianyi Wang, Yiming Wu, Yi Xie, Xiangchao Yan, Jingqi Ye, Peng Ye, Fangchen Yu, Jiakang Yuan, Bihao Zhan, Bo Zhang, Chen Zhang, Shufei Zhang, Shuaiyu Zhang, Wenlong Zhang, Yiqun Zhang, Junpeng Zhao, Zhijie Zhong, Bowen Zhou, and Yuhao Zhou. Scaling the horizon, not the parameters: Reaching trillion-parameter performance with a 35b agent, 2026.
- [9] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [10] Daniel M Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.
- [11] Bo Adler, Niket Agarwal, Ashwath Aithal, Dong H Anh, Pallab Bhattacharya, Annika Brundyn, Jared Casper, Bryan Catanzaro, Sharon Clay, Jonathan Cohen, et al. Nemotron-4 340b technical report. *arXiv preprint arXiv:2406.11704*, 2024.
- [12] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024.
- [13] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations*, volume 2024, pages 39578–39601, 2024.
- [14] Jian Zhao, Runze Liu, Kaiyan Zhang, Zhimu Zhou, Junqi Gao, Dong Li, Jiafei Lyu, Zhouyi Qian, Biqing Qi, Xiu Li, and Bowen Zhou. Genprm: Scaling test-time compute of process reward models via generative reasoning. *arXiv preprint arXiv:2504.00891*, 2025.
- [15] Runze Liu, Junqi Gao, Jian Zhao, Kaiyan Zhang, Xiu Li, Biqing Qi, Wanli Ouyang, and Bowen Zhou. Can 1b llm surpass 405b llm? rethinking compute-optimal test-time scaling. *arXiv preprint arXiv:2502.06703*, 2025.
- [16] Deqing Fu, Tong Xiao, Rui Wang, Wang Zhu, Pengchuan Zhang, Guan Pang, Robin Jia, and Lawrence Chen. Tldr: Token-level detective reward model for large vision language models. In *International Conference on Learning Representations*, volume 2025, pages 45205–45236, 2025.

- [17] Eunseop Yoon, Hee Suk Yoon, Soohwan Eom, Gunsoo Han, Daniel Nam, Daejin Jo, Kyoung-Woon On, Mark Hasegawa-Johnson, Sungwoong Kim, and Chang Yoo. Tlcr: Token-level continuous reward for fine-grained reinforcement learning from human feedback. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 14969–14981, 2024.
- [18] Harrison Lee, Samrat Phatale, Hassan Mansoor, Kellie Ren Lu, Thomas Mesnard, Johan Ferret, Colton Bishop, Ethan Hall, Victor Carbune, and Abhinav Rastogi. Rlaif: Scaling reinforcement learning from human feedback with ai feedback. 2023.
- [19] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- [20] Guanting Dong, Hongyi Yuan, Keming Lu, Chengpeng Li, Mingfeng Xue, Dayiheng Liu, Wei Wang, Zheng Yuan, Chang Zhou, and Jingren Zhou. How abilities in large language models are affected by supervised fine-tuning data composition. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 177–198, Bangkok, Thailand, August 2024. Association for Computational Linguistics.
- [21] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [22] Jianping Gou, Baosheng Yu, Stephen J Maybank, and Dacheng Tao. Knowledge distillation: A survey. *International journal of computer vision*, 129(6):1789–1819, 2021.
- [23] Yoon Kim and Alexander M Rush. Sequence-level knowledge distillation. In *Proceedings of the 2016 conference on empirical methods in natural language processing*, pages 1317–1327, 2016.
- [24] Shicheng Tan, Weng Lam Tam, Yuanchun Wang, Wenwen Gong, Shu Zhao, Peng Zhang, and Jie Tang. Gkd: A general knowledge distillation framework for large-scale pre-trained language model. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 5: Industry Track)*, pages 134–148, 2023.
- [25] Dan Busbridge, Amitis Shidani, Floris Weers, Jason Ramapuram, Etai Littwin, and Russ Webb. Distillation scaling laws. *arXiv preprint arXiv:2502.08606*, 2025.
- [26] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
- [27] Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huan-ang Gao, Wenkai Yang, Zhiyuan Liu, et al. Rethinking on-policy distillation of large language models: Phenomenology, mechanism, and recipe. *arXiv preprint arXiv:2604.13016*, 2026.
- [28] Wenkai Yang, Weijie Liu, Ruobing Xie, Kai Yang, Saiyong Yang, and Yankai Lin. Learning beyond teacher: Generalized on-policy distillation with reward extrapolation. *arXiv preprint arXiv:2602.12125*, 2026.
- [29] Dongxu Zhang, Zhichao Yang, Sepehr Janghorbani, Jun Han, Andrew Ressler II, Qian Qian, Gregory D Lyng, Sanjit Singh Batra, and Robert E Tillman. Fast and effective on-policy distillation from reasoning prefixes. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 25553–25569, 2026.
- [30] Tianzhu Ye, Li Dong, Zewen Chi, Xun Wu, Shaohan Huang, and Furu Wei. Black-box on-policy distillation of large language models. *arXiv preprint arXiv:2511.10643*, 2025.
- [31] Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, et al. Deepmath-103k: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning. *arXiv preprint arXiv:2504.11456*, 2025.
- [32] Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Yuchen Zhang, Jiacheng Chen, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, et al. Process reinforcement through implicit rewards. *arXiv preprint arXiv:2502.01456*, 2025.
- [33] Naman Jain, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, volume 2025, pages 58791–58831, 2025.

- [34] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025.
- [35] Jujie He, Jiakai Liu, Chris Yuhao Liu, Rui Yan, Chaojie Wang, Peng Cheng, Xiaoyu Zhang, Fuxiang Zhang, Jiacheng Xu, Wei Shen, Siyuan Li, Liang Zeng, Tianwen Wei, Cheng Cheng, Bo An, Yang Liu, and Yahui Zhou. Skywork open reasoner 1 technical report. *arXiv preprint arXiv:2505.22312*, 2025.
- [36] Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Huang, Kashif Rasul, Longhui Yu, Albert Q Jiang, Ziju Shen, et al. Numinamath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions. *Hugging Face repository*, 13(9):9, 2024.
- [37] OpenCompass Contributors. Opencompass: A universal evaluation platform for foundation models. <https://github.com/open-compass/opencompass>, 2023.
- [38] Mislav Balunovic, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Matharena: Evaluating llms on uncontaminated math competitions. *Advances in Neural Information Processing Systems*, 38, 2026.
- [39] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [40] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [41] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

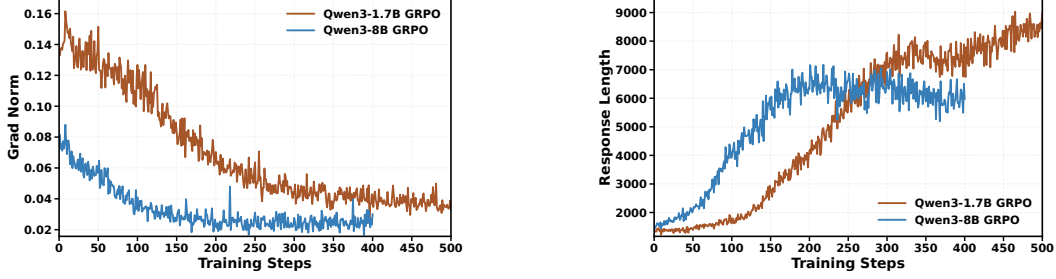


Figure 7: Left: Gradient norm during GRPO training of the proxy expert models. Right: Response length during GRPO training of the proxy expert models.

Appendix

Contents

A Experiment Details	14
A.1 Implementation Details	14
A.2 Effect of the Calibration Coefficient λ	15

A Experiment Details

A.1 Implementation Details

Models and framework. All experiments are conducted on the Qwen3 family [9] (1.7B, 4B, 8B), all operating in non-thinking mode. Our implementation is built upon verl [12], and all training is performed on 8×NVIDIA A100 80GB.

Proxy exploration. The proxy experts are trained with GRPO until convergence; hyper-parameters are listed in Table 4. We train on DeepMath-103K [31] for the math domain (filtered following prior work) and on Eurus-RL-Code [32] for the code domain, using a rule-based verifiable reward (1 for correct answers and 0 otherwise).

Hyper-param.	Value
Train Batch Size	128
Rollout n	8
PPO Mini Batch Size	128
Max Response Length	16,384
Temperature	1.0
Top- p	1.0
LR	1×10^{-6}
KL Coefficient	0.0

Table 4: Training hyper-parameters of GRPO in math and code RL.

Signal transfer (P-OPD). The training hyper-parameters for signal transfer are summarized in Table 5. The calibration coefficient λ is set as described in Section 5.3.

Evaluation protocol. For math benchmarks, we report Mean@16 (the average accuracy over 16 independently sampled responses) across a comprehensive suite of eight datasets: AIME24 [36], AIME25 [37], AIME26, CMIMC25, HMMT25(Feb) [38], HMMT25(Nov) [38], HMMT26(Feb), and SMT25 [31]. Due to space constraints, the main text reports a representative subset of four benchmarks, while the full results on all eight datasets are detailed in the appendix. For code benchmarks, i.e. HumanEval+ [39, 40], MBPP+ [41] and LCB [33], we report Mean@8 and strictly follow the evaluation scripts of [28].

Hyper-parameter	Value
Train Batch Size	256
Rollout n	8
PPO Mini Batch Size	1024
Max Response Length	16,384
Temperature	1.0
Top- p	1.0
LR	1×10^{-6}

Table 5: Training hyper-parameters of P-OPD signal transfer on math and code domain.

To ensure that the update signal transferred by P-OPD is both reliable and high-quality, we train each proxy expert model with GRPO until full convergence, rather than terminating at a fixed budget. The complete set of training hyper-parameters is summarized in Table 4, which we keep consistent across all proxy model scales.

Figure 7 (left) and Figure 7 (right) present the training dynamics of the proxy experts (Qwen3-1.7B and Qwen3-8B) throughout the GRPO process. As shown in Figure 7 (left), the gradient norm decreases steadily and stabilizes at a low plateau (~ 0.03) in the later training phase, indicating that the optimization has reached a stable, converged regime. Meanwhile, Figure 7 (right) shows that the response length grows and gradually saturates, reflecting that the model has developed stable reasoning behaviors rather than still actively evolving its output distribution. Together, these curves confirm that our proxy experts are trained to convergence, providing a well-formed update signal for cross-scale transfer.

A.2 Effect of the Calibration Coefficient λ

As discussed in “Method” section, the calibration coefficient λ controls how strongly the target model compensates for the update it has already absorbed: a smaller λ follows the proxy-induced direction more aggressively, while a larger λ yields more conservative transfer. To understand its practical effect, we sweep λ when transferring update signals to the same 8B target model from two proxy experts of different scales: 4B proxy and 1.7B proxy. We report validation accuracy, policy entropy, gradient norm, and training loss throughout transfer.

4B→8B transfer. Figures 8a–8d and Table 6 show the training dynamics when the 4B proxy guides the 8B model. All configurations improve rapidly in the early steps, but the smallest coefficient ($\lambda = 0.5$) is clearly too aggressive: its validation accuracy peaks early and then degrades, and its entropy stays the lowest, indicating premature over-updating. Notably, setting $\lambda = 0$ leads to a complete performance collapse, which is likely due to the compensation coefficient being too low to prevent over-optimization. In contrast, moderate-to-large coefficients ($\lambda \in \{0.75, 0.95, 1.0, 1.5\}$) remain stable and converge to consistently higher accuracy, with entropy and gradient norm settling into a smooth plateau. This suggests that a sufficient amount of calibration is needed to avoid over-following the fixed proxy direction.

1.7B→8B transfer. Figures 9a–9d and Table 7 report the results when the signal comes from the smaller 1.7B proxy. The overall accuracy is lower than in the 4B proxy case, which is expected given the larger capability gap between proxy and target. Here the trend with respect to λ is even sharper: the most conservative setting ($\lambda = 1.5$) achieves the best and most stable accuracy, while $\lambda = 0.5$ directly leads to a performance collapse. This severe degradation further confirms that an excessively low compensation coefficient cannot effectively regularize the noisier proxy direction, requiring stronger calibration.

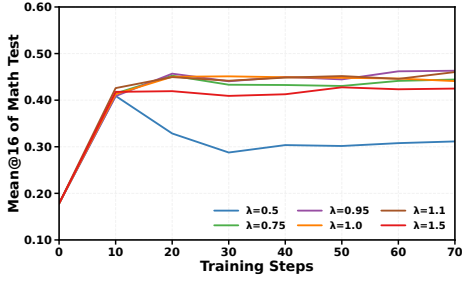
Across both settings, the training loss and gradient norm converge stably for all λ , confirming that the transfer objective is well-behaved. The key takeaway is that overly aggressive transfer (small λ) consistently hurts performance, and the optimal degree of calibration increases as the proxy–target gap widens.

Table 6: Mean@16 results on different math benchmarks of P-OPD (4B→8B). Performance comparison across calibration coefficients λ . The best results are highlighted in **bold**.

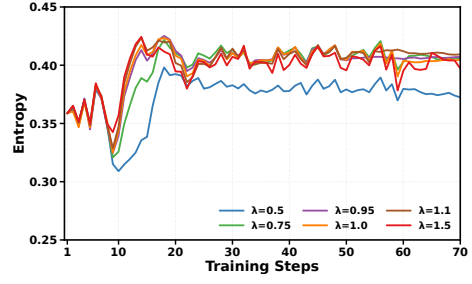
Benchmark	λ						
	0	0.5	0.75	0.95	1.0	1.1	1.5
AIME24	0	57.29	61.25	60.83	62.50	63.54	60.42
AIME25	0	41.88	50.63	50.21	52.71	52.92	48.33
CMIMC25	0	8.82	27.81	29.22	28.44	29.53	31.88
AIME26	0	45.63	56.88	56.67	56.67	55.63	52.29
HMMT25(Feb)	0	23.75	34.17	31.88	34.17	32.08	27.92
HMMT25(Nov)	0	26.67	37.71	41.04	40.42	40.83	37.50
SMT25	0	12.31	52.48	52.48	50.94	52.01	19.76
HMMT26(Feb)	0	30.43	32.20	33.33	33.52	34.85	31.82
AVG.	0	30.85	44.14	44.46	44.92	45.17	38.74

Table 7: Mean@16 results on math benchmarks of P-OPD (1.7B→8B). Performance comparison across calibration coefficients λ . The best results are highlighted in **bold**.

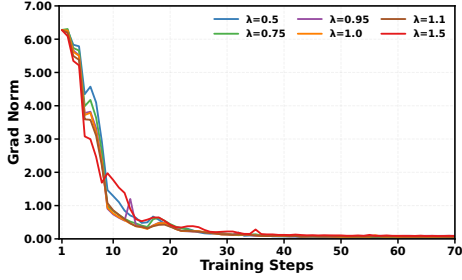
Benchmark		λ					
		0.5	1.0	1.25	1.5	1.75	2.0
AIME24	0	41.04	50.83	54.38	51.67	47.71	
AIME25	0	31.88	37.92	40.21	39.17	36.88	
CMIMC25	0	16.41	22.50	26.41	25.47	25.00	
AIME26	0	32.29	43.13	46.25	43.13	36.88	
HMMT25(Feb)	0	13.54	19.58	22.08	22.50	21.88	
HMMT25(Nov)	0	25.00	29.17	31.25	30.00	29.38	
SMT25	0	35.85	42.93	48.23	44.10	42.22	
HMMT26(Feb)	0	24.05	27.65	28.60	29.55	28.98	
AVG.	0	27.51	34.21	37.19	35.70	33.61	



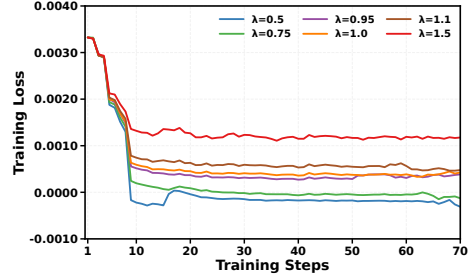
(a) Mean@16 of Math Test under different calibration coefficient λ (4B $\rightarrow$ 8B).



(b) Conditioned entropy of target model under different λ (4B $\rightarrow$ 8B).

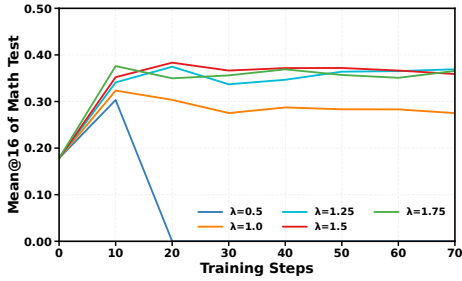


(c) Gradient norm under different calibration coefficient λ (4B $\rightarrow$ 8B).

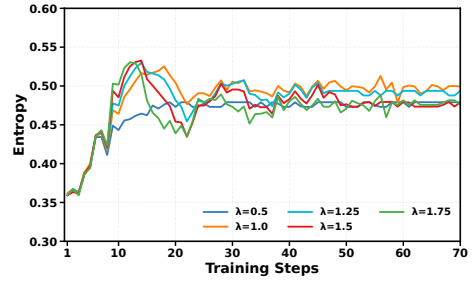


(d) Training loss under different calibration coefficient λ (4B $\rightarrow$ 8B).

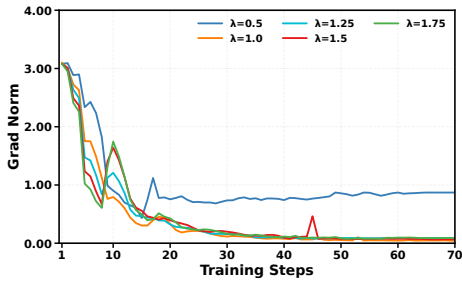
Figure 8: Effects of different calibration coefficients λ for the 4B $\rightarrow$ 8B setting.



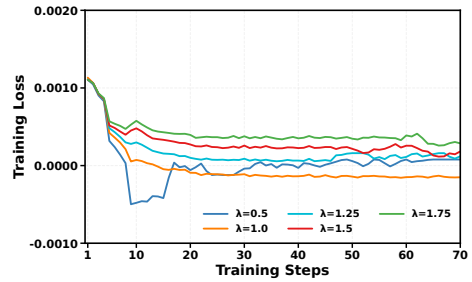
(a) Validation accuracy under different calibration coefficient λ (1.7B $\rightarrow$ 8B).



(b) Conditioned entropy of target model under different λ (1.7B $\rightarrow$ 8B).



(c) Gradient norm under different calibration coefficient λ (1.7B $\rightarrow$ 8B).



(d) Training loss under different calibration coefficient λ (1.7B $\rightarrow$ 8B).

Figure 9: Effects of different calibration coefficients λ for the 1.7B $\rightarrow$ 8B setting.